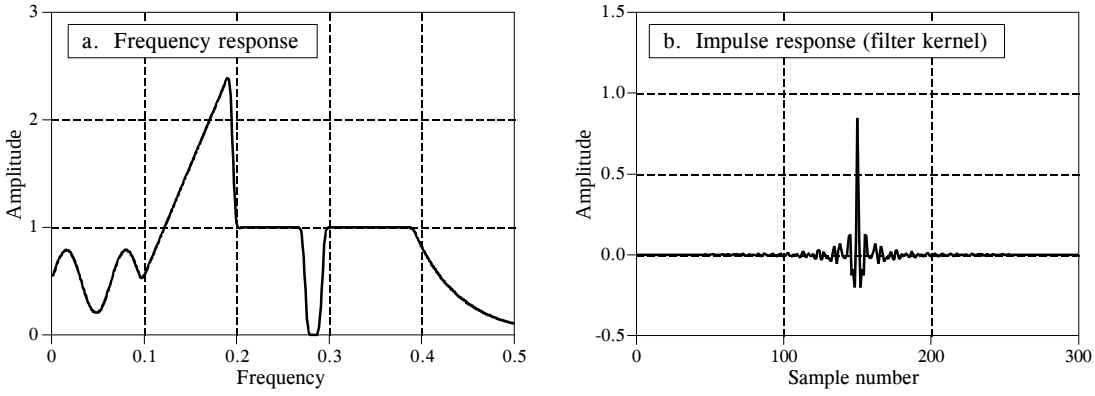

FIGURE 29-2
Example FIR filter. In (a) the frequency response of a highly custom filter is shown. The corresponding impulse response (filter kernel) is shown in (b). This filter was designed in Chapter 17 to show that virtually *any* frequency response can be achieved with FIR digital filters.

Since the source code is in ASCII, a standard text editor is all that is needed to make changes to existing files, or create entirely new programs. Table 29-2 shows an example of an FIR filter program written in assembly. While this is the only code you need to worry about for now, keep in mind that there are other files needed to make this a complete program. This includes an "architecture description file" (which defines the hardware configuration and memory allocation), setup of the interrupt vector table, and a codec initialization routine. Eventually you will need to understand what goes on in these sections, but for now you simply copy them from the prewritten programs.

As shown at the top of Table 29-2, there are three variables that need to be defined before jumping into the main section of code. These are the number of points in the filter kernel, **NR_COEF**; a circular buffer that holds the past samples from the input signal, **dline[ ]**; and a circular buffer that holds the filter kernel, **coef[ ]**. We also need to give the program two other pieces of information: the sampling rate of the codec, and the name of the file containing the filter kernel, so that it can be read into coef[ ]. All these steps are easy; nothing more than a single line of code each. We don't show them in this example because they are contained in the sections of code that we are ignoring for simplicity.

Figure 29-2 shows the filter kernel we will test the program with, the same custom filter we designed in Chapter 17. As you recall, this filter was chosen to have a very irregular frequency response, reinforcing the notion that FIR digital filters can provide virtually any frequency response you desire. Figure (a) shows the frequency response of our test filter, while (b) shows the corresponding impulse response (i.e., the filter kernel). This 301 point filter kernel is stored in an ASCII file, and is combined with the other sections of code during linking to form a single executable program.